

Detection of Prescribed Upper Uniform Compliance Among NU Lipa Students

***Development and Implementation of a YOLO-Based Object Detection Model Using
Python***

**A YOLO Project
School of Architecture, Computing and Engineering
NU Lipa, Marawoy, Lipa City**

**In Partial Fulfillment of the Requirements
For the Bachelor of Science in Computer Science
With Specialization in Machine Learning**

**Submitted by:
Guce, Ceile Marie
Pranada, Emmanuelle P.
COM231**

**Instructor:
Joel S. Enriquez**

October 9, 2026

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

School uniforms are a visible sign of discipline, identity, and belonging in Philippine schools, and wearing them correctly is a basic expectation of students. National University Lipa (NU Lipa) in Batangas follows the National University Student Handbook for AY 2024–2025, which requires students to wear the prescribed uniform inside the school premises from Mondays through Fridays and during official University activities outside the campus, unless specific instructions are given (Section I.E; National University, 2024).

For most programs, the prescribed uniform is a light blue polo with the National University logo for male students, or a light blue blouse with the logo for female students, worn with navy blue slacks (or a skirt for female students) and black leather shoes. Students of Allied Health, Dentistry, Tourism, and Hospitality Management follow the uniform guidelines of their own colleges. Wednesdays and Saturdays are washdays, when the prescribed uniform is not required unless the College Dean instructs otherwise, and students may wear their PE or NSTP uniform only under the conditions set in the handbook. In this report, the light blue polo or blouse with the University logo is called the traditional upper uniform. [Confirm that this matches your definition and add any NU Lipa-specific rules.]

The handbook classifies the non-wearing of the prescribed uniform inside the campus as a Minor Offense (Section VI.D): the first offense results in a violation slip from the Student Discipline Office, the second adds a written warning, and non-compliance becomes a **Major Offense** on its third occurrence unless the student has a written permit from the office (Sections I.E and VI.D; National University, 2024). Because violations are counted by occurrence, consistent and well-documented checking matters.

Checking uniform compliance is usually done by hand: guards, teachers, or student officers look at each student at the gate or in the corridors. Studies on campus dress-code monitoring report that campus security still enforces clothing rules manually (Tantra & Widjaja, 2025), and a review of dress-code violation detection in educational environments notes that automated systems can reduce human error and bias in enforcement (see Appendix F). Manual checks also take time away from teachers and security staff, especially during peak entry hours.

Computer vision offers a way to support this task. Object detection models find objects in images or video and label them in a single pass. You Only Look Once (YOLO), first introduced by Redmon et al. (2016), is a family of real-time detectors that is widely used because it is fast enough to run on

video. Researchers have already applied YOLO to student uniform checking, for example in dress-code surveillance (Tantra & Widjaja, 2025), school uniform checking together with attendance monitoring (Lam et al., 2023), and uniform detection combined with face recognition (Basukala et al., 2025).

This project develops a YOLO-based object detection model that focuses on one specific rule at NU Lipa: the wearing of the traditional upper uniform. The model detects each student's upper body and decides whether the traditional upper uniform is worn (`in_uniform`) or not worn (`not_uniform`), where not wearing the prescribed uniform is the violation defined in the handbook. Limiting the scope to one clearly defined garment keeps the problem well defined, makes annotation consistent, and suits an individual project with limited time and hardware.

1.2 Problem Statement

Uniform compliance at NU Lipa is currently verified through manual inspection, which is time-consuming, can be inconsistent between inspectors, and is difficult to record or review later. There is no automated tool that recognizes whether a student is wearing the traditional upper uniform in images or video taken on campus.

Specifically, this project seeks to answer the following questions:

1. How can an image dataset of students wearing and not wearing the traditional upper uniform be collected, cleaned, and annotated for YOLO training?
2. How accurately can a YOLO model detect `in_uniform`, and `not_uniform` as measured by precision, recall, mAP@0.50 , and mAP@0.50:0.95 ?
3. How does a training parameter the YOLO model size affect the model's performance?
4. What are the strengths, limitations, and common errors of the model when tested on unseen images, video, or webcam input?

1.3 Objectives

General objective. To develop and implement a YOLO-based object detection model using Python that detects whether students at NU Lipa are wearing the traditional upper uniform.

Specific objectives. The project aims to:

- Explain the fundamental concepts of YOLO and object detection relevant to uniform compliance monitoring.

- Identify uniform compliance detection as a real-world object detection problem at NU Lipa and define the target classes.
- Collect, select, and prepare an image dataset of at least 500 images with at least two classes, divided into training, validation, and test sets.
- Annotate the images with bounding boxes and class labels compatible with YOLO using Computer Vision Annotation Tool (CVAT) and Roboflow
- Configure and train a YOLO model using Python and the Ultralytics library.
- Evaluate the trained model using precision, recall, mAP@0.50, mAP@0.50:0.95, and a confusion matrix.
- Test the model on unseen images, video, or webcam input.
- Analyze the strengths, limitations, and errors of the developed model.
- Document the development process in a technical report and present a working demonstration.

1.4 Scope and Limitations

Scope.

- The system detects only the traditional upper uniform worn by students of NU Lipa. Other items (lower uniform, shoes, ID, hair, bag, accessories) are outside the scope.
- Two classes are used: **in_uniform (traditional upper uniform worn)** and **not_uniform** (violation: traditional upper uniform not worn, following Section VI.D of the Student Handbook and Appendix B, Table B.2).
- The dataset contains 504 images, divided into 379 training images (75.2%), 75 validation images (14.9%), and 50 test images (9.9%).
- The model is built with YOLO in Python and demonstrated on webcam.

Limitations.

- The model works on images and video only and does not identify individual students or keep a record of who committed a violation.
- Performance depends on the conditions covered by the dataset (lighting, camera angle, distance, and crowding) and may drop outside them.
- The dataset comes from multiple online NU publications, so it may not represent every uniform variant, body size, or season.
- The model's output is for decision support and is not connected to any official disciplinary process.
- The model cannot know about the exceptions allowed in the handbook: washdays (Wednesday and Saturday), PE or NSTP uniforms in permitted cases, programs with their own uniforms, and written permits from the Student

Discipline Office. Images are therefore collected and labeled under the conditions in Appendix B, and every flagged case needs a human check.

- Training was limited by the available hardware: 4 vCPUs, 2 × NVIDIA Tesla T4 GPUs (reported VRAM: 30 GB total; verify allocation).

1.5 Significance of the Project

Students: a consistent and objective way of checking compliance that does not depend on who is on duty.

Guidance, Student Affairs, and security personnel: faster screening of images or video, so staff can focus on cases that need human attention.

School administration: a low-cost prototype that shows how computer vision can support campus discipline policies.

Future researchers: a documented dataset, methodology, and baseline results at NU Lipa that can be improved, for example with more classes or deployment on cameras.

The researcher: practical experience in dataset preparation, annotation, model training, evaluation, and error analysis.

CHAPTER 2

RELATED TECHNOLOGIES

2.1 Computer Vision

Computer vision is the field of artificial intelligence that enables computers to interpret images and video. Typical tasks include image classification (what is in the picture), object detection (what is in the picture and where), segmentation (which pixels belong to which object), and tracking. Modern computer vision systems rely mainly on deep learning, in which neural networks learn visual features from large sets of labeled examples instead of relying on hand-written rules. In this project, computer vision is used to turn campus images into a decision: is the student wearing the traditional upper uniform or not.

2.2 Object Detection

Object detection locates every object of interest in an image by drawing a bounding box around it and assigning a class label with a confidence score. Two measures appear throughout this report. Intersection over Union (IoU) measures how much a predicted box overlaps the ground-truth box. Non-maximum suppression (NMS) removes duplicate boxes for the same object. Detectors are commonly grouped into two-stage methods, which first propose regions and then classify them (for example, Faster R-CNN, used for dress-code violation detection in a study listed in Appendix F), and one-stage methods such as YOLO, which predict boxes and classes in one pass. One-stage detectors are generally preferred when speed matters, such as in video monitoring.

2.3 YOLO

YOLO (You Only Look Once) was introduced by Redmon et al. (2016) as a detector that frames detection as a single regression problem, so one neural network predicts bounding boxes and class probabilities directly from the full image. Because it looks at the whole image once, it is fast enough for real-time use. The approach has been developed through many versions since then, and the Ultralytics library now provides a common Python interface for training, evaluating, and running YOLO models.

This project uses YOLO11, released by Ultralytics on 27 September 2024. According to the Ultralytics documentation (Ultralytics, n.d.-a), YOLO11 has an improved backbone and neck for better feature extraction, supports detection, segmentation, classification, pose estimation, and oriented boxes, and comes in five sizes: nano (n), small (s), medium (m), large (l), and extra-large (x).

Table 2.1. YOLO11 model sizes (qualitative comparison)

Size	Name	Typical use
------	------	-------------

Size	Name	Typical use
n	Nano	Fastest and lightest; small datasets, CPUs, edge devices
s	Small	Balance of speed and accuracy
m	Medium	Higher accuracy; needs a stronger GPU
l / x	Large / Extra-large	Highest accuracy; slowest and most memory-hungry

2.4 Convolutional Neural Networks

A convolutional neural network (CNN) is a neural network designed for images. Convolutional layers slide small filters over the image to detect simple patterns such as edges and textures, and deeper layers combine them into more complex patterns such as collars, shirt shapes, and body parts. Pooling or strided convolutions reduce the image size while keeping the important features, and the final layers use these features to make predictions. YOLO models are built from CNN components arranged in three parts: a backbone that extracts features, a neck that combines features from different scales so objects of different sizes can be detected, and a head that outputs the boxes and class scores. Because the network learns its own features from labeled images, a model trained on enough varied examples of the uniform can learn to recognize it without hand-written rules.

2.5 Dataset Annotation

Annotation is the process of marking each target object in every training image with a bounding box and a class label, and it is how the model is told what to learn. Annotation quality directly limits model quality: loose boxes, missed objects, or inconsistent labeling rules lead to weaker detection. YOLO uses a text file for each image, with one line per object containing the class number and the box center x, center y, width, and height as values normalized between 0 and 1. The file for an image has the same name as the image and is stored in the matching labels folder. Tools such as Roboflow, CVAT, LabelImg, and Label Studio can draw the boxes and export labels in YOLO format. This project used CVAT (see Section 3.3).

2.6 Python

Python is a high-level programming language that is the standard language for machine learning because of its readable syntax and large ecosystem of libraries for numerical computing, data handling, plotting, and deep learning. All training, evaluation, and demonstration scripts in this project are written in Python [3.11].

2.7 Selected Libraries and Tools

Table 2.2. Libraries and tools used in the project

Library / tool	Purpose in this project
Ultralytics YOLO	Loading pretrained YOLO models, training, validation, and prediction
PyTorch	Deep learning framework that runs the model, with optional GPU acceleration
OpenCV	Reading images, video, and webcam frames; drawing results on frames
NumPy	Numerical arrays used by the image and model pipeline
Pandas	Reading results.csv and organizing experiment results into tables
Matplotlib	Plotting training curves and charts
CVAT & Roboflow	Drawing bounding boxes and exporting YOLO-format labels
Kaggle	Writing and running the code
Git / GitHub	Version control and sharing of code

2.8 Related Studies

Several studies have applied object detection to uniform or dress-code monitoring. Table 2.3 summarizes the ones reviewed for this project.

Table 2.3. Summary of related studies

Study	Approach	Notes
Tantra & Widjaja (2025)	YOLOv8 for detecting clothing allowed or disallowed on a university campus	Reported an F1-score of 45% and 51.8% mAP; the authors described the model as satisfactory with room for improvement.
Lam et al. (2023)	Six YOLOv8-based models for attendance and uniform checking (shirts, pants, faces, ID cards)	Performed fairly well under optimal lighting conditions.
Basukala et al. (2025)	YOLOv8 for student uniform parts (shirts, ties, pants) with face recognition	Used data augmentation to address lighting variation.

Study	Approach	Notes
Dress-code violation study (AJERD)	Faster R-CNN for detection and EfficientNet for classification	Presented automation as a way to reduce bias and human error.
Bulletin of Information Technology (2026)	YOLOv8n with data augmentation for student dress-code surveillance	Focused on improving the accuracy of a compact model.
gisketch (n.d.)	YOLOv8 software for complete or incomplete uniforms at the University of Southern Mindanao	Logs detection time, gender, and uniform completeness.
STIS proceedings paper	YOLO to detect completeness of academic uniform attributes	Built on a dataset collected by the researchers.

Most of these studies detect several garments or add face recognition, and none targets the specific uniform rules of NU Lipa, to the extent of the sources found. This project therefore focuses on one clearly defined item, the traditional upper uniform, and evaluates it with the metrics required by the course.

CHAPTER 3 METHODOLOGY

3.1 Project Development Process

The project followed the development flow recommended in the project guideline, from problem identification to application demonstration. Table 3.1 summarizes each stage and its output.

Table 3.1. Project development process

Step	Activity	Output
1. Problem identification	Defined the uniform compliance problem and the two classes based on Sections I.E and VI.D of the NU Student Handbook (AY 2024–2025)	Problem statement and class definitions
2. Dataset collection	Collected images of students with and without the traditional upper uniform at multiple online sources	504 raw images
3. Data cleaning	Removed blurred, corrupt, and duplicate images	Clean image set
4. Image annotation	Drew bounding boxes and class labels using CVAT	YOLO-format label files
5. Dataset preparation	Split the data into train, validation, and test sets; wrote data.yaml	Dataset folders and data.yaml
6. YOLO configuration	Selected the model, image size, batch size, and epochs	Training configuration
7. Model training	Trained with Ultralytics in Python	best.pt weights and training curves
8. Model evaluation	Evaluated on the validation set	Metrics table and confusion matrix
9. Model testing	Ran detection on unseen images, video, or webcam	Prediction images and examples
10. Application demonstration	Ran the working inference program	Live demonstration

3.2 Dataset Description

The dataset contains 504 images gathered from online sources and/or campus sources as described in the project records. Images vary in lighting, camera angle, distance, and

number of people per image. The source-level collection details should be completed in Appendix B before submission.

Table 3.2. Dataset summary

Item	Description
Total images	504
Classes	in_uniform, not_uniform
Training set (75.2%)	379 images
Validation set (14.9%)	75 images
Test set (9.9%)	50 images
Class distribution	in_uniform: 916 instances; not_uniform: 1067 instances
Image quality	Differing image qualities in differing lighting conditions
How the split was made	Randomized split utilizing a script

3.3 Data Annotation

Every target object in every image was annotated with a bounding box and a class label using CVAT. Each box was drawn around the visible upper-body uniform area of one student and was labeled `in_uniform` or `not_uniform` using the definitions in Appendix B. Labels were exported in YOLO format (one .txt file per image). To keep labeling consistent, a written labeling guide was used, and a sample of labels was re-checked.

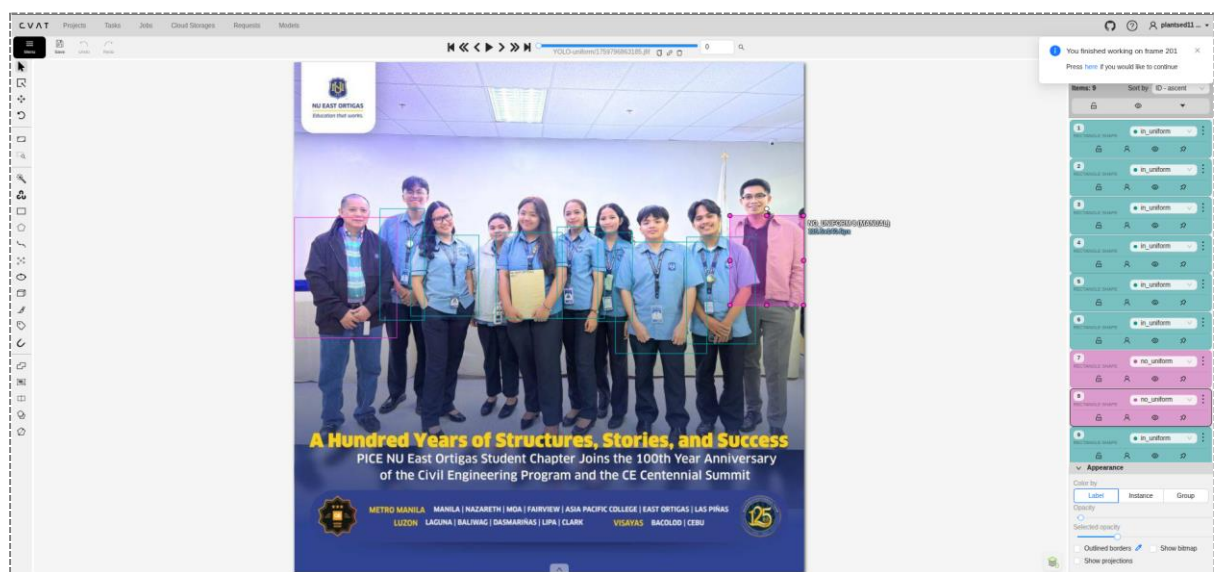


Figure 3.1. *Example of an annotated training image with bounding boxes and class labels.*

3.4 Model Configuration

The model was configured as follows. YOLO11n was chosen as the final model because the dataset is small, a compact model is less likely to overfit it, and it trains and runs quickly on the available hardware, which also leaves room for real-time use. The comparison in Section 4.3.1 supports this choice, because YOLO11n matched the accuracy of the larger models while training the fastest. The model started from pretrained weights (yolo11n.pt) and was fine-tuned on the project dataset, which is called transfer learning and usually works better than training from scratch on a small dataset. The dataset paths and class names were defined in data.yaml (Appendix A.2). The main settings were an image size of 736, a batch size of 16, and Ultralytics default augmentation unless stated otherwise in Table 4.2. Training used a two-phase approach that is treated as one continuous session. In Phase 1 (head), only the detection head was trained while the backbone stayed frozen, with a configured maximum of 100 epochs. In Phase 2 (full), all layers were unfrozen and the whole model was fine-tuned, continuing from Phase 1, with a configured maximum of 500 epochs. Training ended before these maximums, after 14 to 58 head epochs and 44 to 96 full epochs depending on the model size (Table 4.8).

3.5 Training Process

Training was performed with the Ultralytics Python API using the script in Appendix A.3 on Kaggle. During training, the training and validation losses and the validation metrics were recorded after every epoch in results.csv, and the checkpoint with the best validation fitness was saved as best.pt. A fixed random seed 42 was used so that results can be reproduced.

To investigate how a parameter affects performance, a controlled experiment was run with the script in Appendix A.7 in which only the YOLO model size was changed. Five YOLO11 variants (n, s, m, l, and x) were each trained with the same two-phase procedure, while the dataset split, image size, batch size, and seed stayed the same.

3.6 Testing Procedure

After training, the final model was evaluated on the validation set, which was not used for training or for selecting best.pt. Table 3.3 defines the metrics reported in Chapter 4, where TP, FP, and FN are true positives, false positives, and false negatives.

Table 3.3. Evaluation metrics

Metric	Definition	Meaning in this project
--------	------------	-------------------------

Metric	Definition	Meaning in this project
Precision	$TP / (TP + FP)$	Of the detections made, the share that were correct
Recall	$TP / (TP + FN)$	Of the real uniforms or violations, the share that were found
F1-score	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	Balance of precision and recall
IoU	Overlap area / Union area of predicted and true boxes	How well the box fits the object
mAP@0.50	Mean of the per-class average precision at IoU 0.50	Overall detection quality with a loose box requirement
mAP@0.50:0.95	mAP averaged over IoU thresholds from 0.50 to 0.95	Overall quality including how tight the boxes are
Confusion matrix	Counts of predicted versus actual classes	Shows which classes are confused or missed

The model was then demonstrated on previously unseen data utilizing a webcam with a confidence threshold of 0.50, using the scripts in Appendices A.5 and A.6. At least 10 representative predictions were recorded, and the errors were reviewed and grouped by type to produce the error analysis in Section 4.5.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the training results, evaluation metrics, detection results, and error analysis of the YOLO-based uniform compliance detection model developed for NU Lipa. The model detects the traditional upper uniform that students are required to wear properly while on campus and labels each detected student as compliant (in_uniform) or in violation (not_uniform).

4.1 Training Results

4.1.1 Hardware and Software Configuration

Table 4.1 lists the hardware and software configuration actually used to train and test the model.

Table 4.1. Hardware and software configuration

Component	Specification
Processor (CPU)	4 vCPUs
Memory (RAM)	
Graphics card (GPU)	2 × NVIDIA Tesla T4 (30 GB VRAM)
Operating system	Not reported in supplied records
Training platform	Kaggle
Python version	3.11
PyTorch version / CUDA	
Ultralytics version	
OpenCV / NumPy / Pandas / Matplotlib	
Annotation tool	CVAT and Roboflow

4.1.2 Training Parameters

Table 4.2. Training parameters of the final model

Parameter	Value used
YOLO model / pretrained weights	yolo11n.pt (COCO pretrained)
Epochs	154 (Phase 1: 58, Phase 2: 96) (best epoch: 129)
Image size (imgsz)	736
Batch size	16
Optimizer / learning rate	AdamW; Phase 1 lr0 = 2e-3; Phase 2 lr0 = 5e-4

Parameter	Value used
Early-stopping patience	Stage 1: 10, Stage 2: 25
Data augmentation	Ultralytics defaults
Random seed	42
Total training time	11.1 minutes (both phases)

4.1.3 Training and Validation Curves

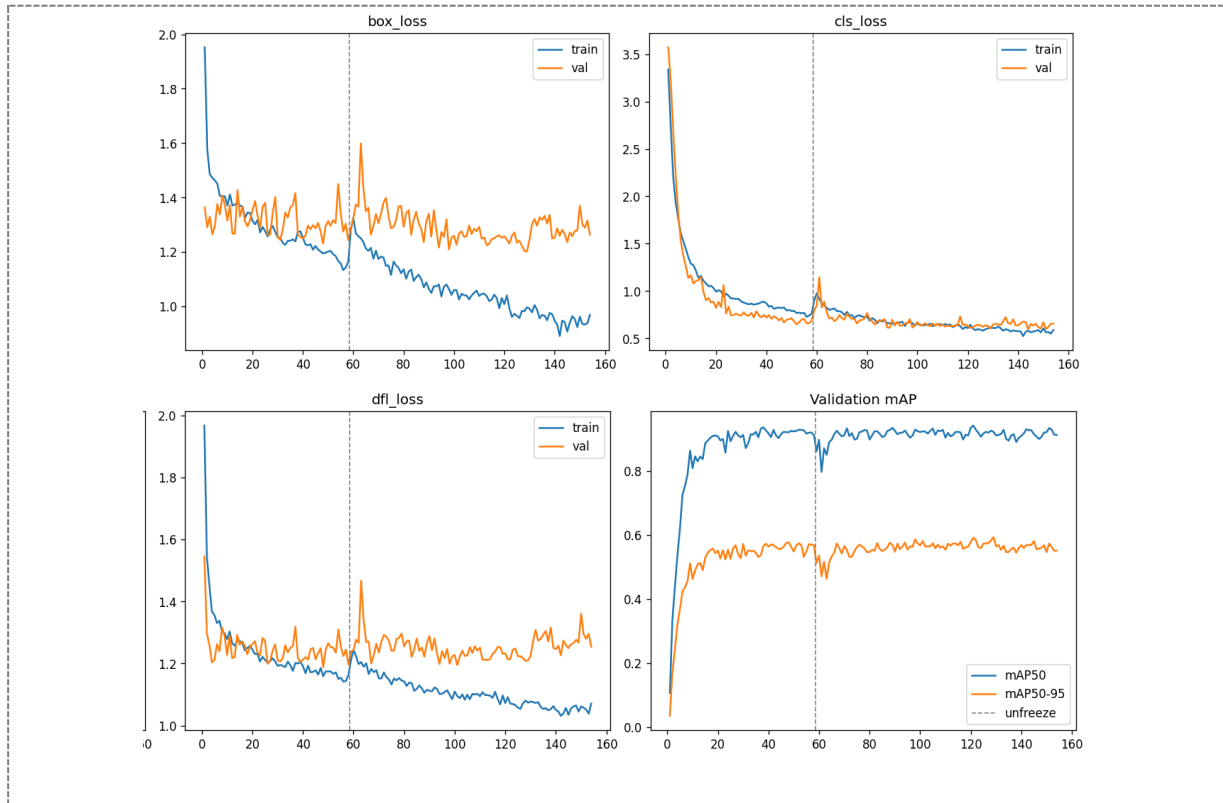


Figure 4.1. Training and validation loss and detection metrics across epochs. Lower loss is generally better; higher precision, recall, and mAP are generally better.

Figure 4.1 contains training-loss curves and validation-loss/metric curves plotted across epochs. Box loss measures bounding-box localization error; classification loss measures class-prediction error; and distribution-focal loss (DFL) helps refine box-coordinate distributions. Lower loss is generally better. The training box, classification, and DFL losses decreased from 1.95, 3.34, and 1.97 at epoch 1 to 0.97, 0.59, and 1.07 at the final epoch. Validation losses improved quickly early in training and then flattened. From the end of the head-only phase to the final epoch, validation box loss changed from 1.24 to 1.26 and validation DFL from 1.20 to 1.25, while training box and DFL losses continued to fall (1.16 to 0.97 and 1.16 to 1.07). This divergence is consistent with a possible mild overfitting tendency: the model continued fitting the training data without corresponding improvement on validation data. Validation mAP@0.50 rose from 0.107 at epoch 1 to 0.936 within the head phase, reached its peak of 0.9417 at epoch 121, and stayed between roughly 0.91 and

0.93 in the sampled epochs after epoch 68 (0.9124 at the last epoch). Validation mAP@0.50:0.95 peaked at 0.5935 at epoch 129. The best weights (best.pt) were saved by validation fitness and used for all subsequent evaluation.

4.2 Confusion Matrix

Figures 4.2–4.3 show the confusion matrix for the validation set. In this report's table convention, rows are predicted classes and columns are actual (ground-truth) classes. The diagonal cells are correct class matches; off-diagonal cells are class confusions. The background row represents ground-truth objects that were missed, while the background column represents predictions that did not match a ground-truth object (false alarms). The normalized matrix expresses these relationships as proportions, making class-wise patterns easier to compare when class counts differ.

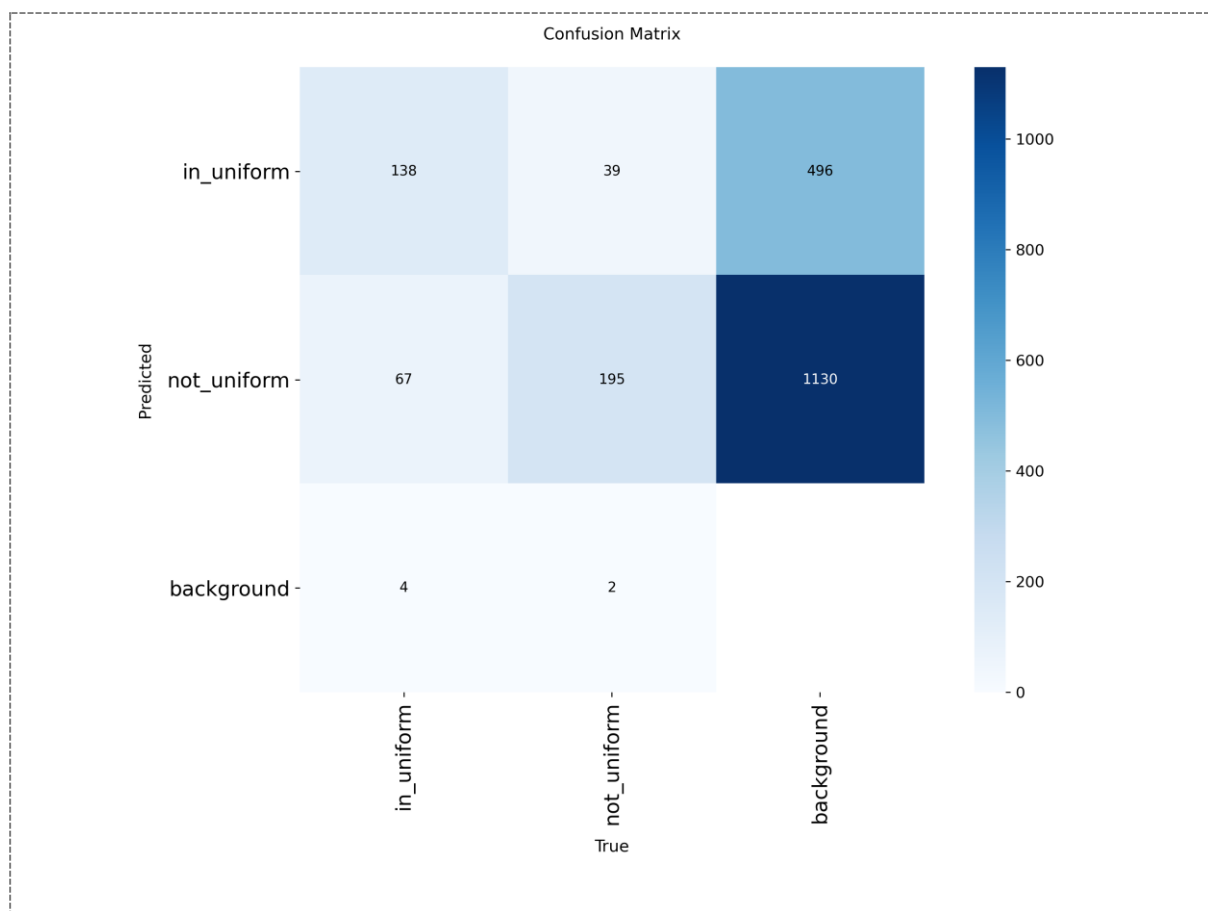


Figure 4.2. Validation-set confusion matrix (counts). Diagonal cells are correct class matches; off-diagonal cells are class confusions; background cells indicate missed objects or unmatched detections.

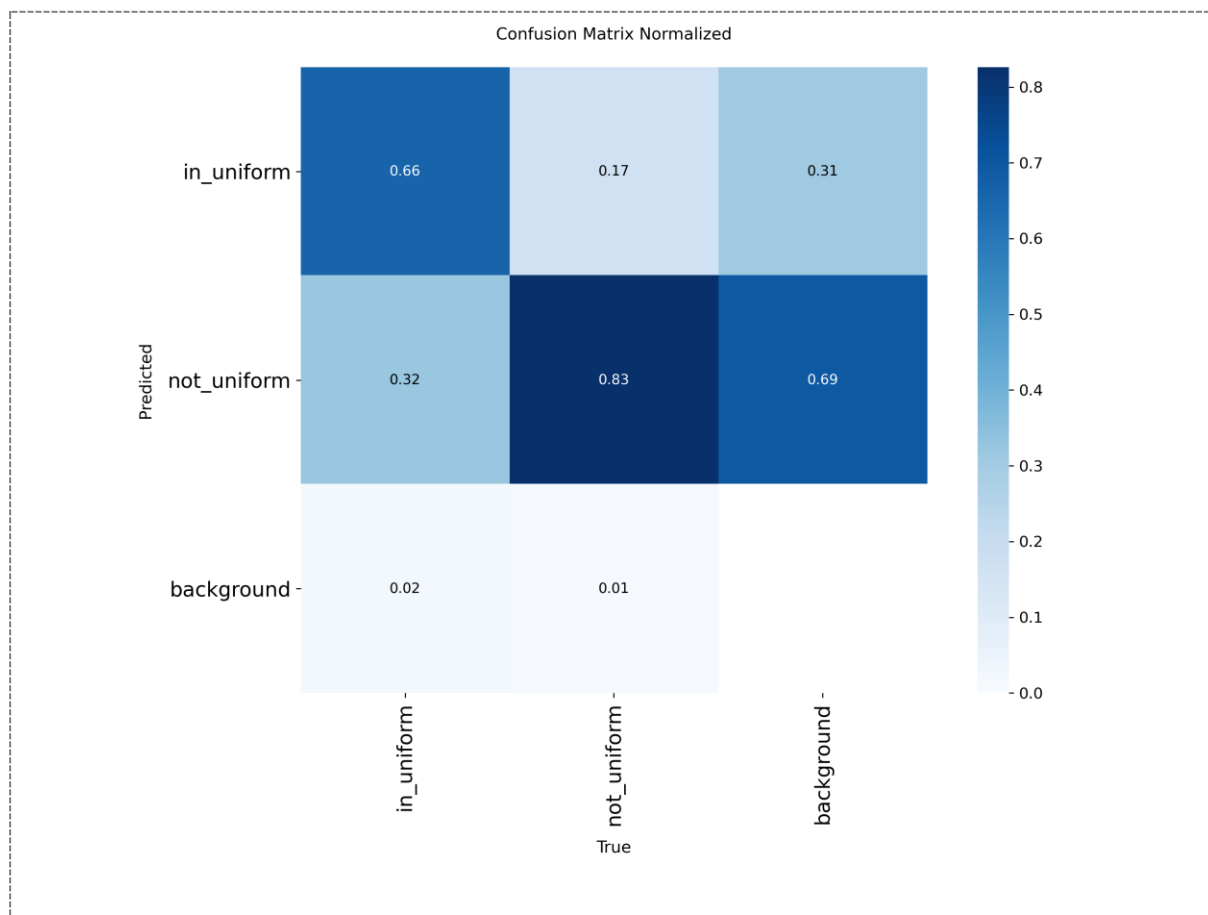


Figure 4.3. Normalized validation-set confusion matrix. Normalization expresses the counts as proportions, making class-wise error patterns easier to compare.

Table 4.3. Confusion matrix counts on the validation set (rows = predicted, columns = actual)

Predicted \ Actual	in_uniform	not_uniform	background (no real object)
in_uniform	138	39	496
not_uniform	67	195	1,130
background (missed)	4	2	—

Table 4.4. What each error means for uniform compliance monitoring

Error	Meaning in this system	Consequence
Actual not_uniform → predicted in_uniform, or missed	Violation not detected (false negative)	A student in violation passes unflagged; reduces the usefulness of the system.
Actual in_uniform → predicted not_uniform	Wrong violation alert (false positive)	A compliant student is wrongly flagged; unfair if acted on without human checking.
Background → not_uniform / in_uniform	False alarm on a non-uniform object or non-student	Extra noise for the reviewer.

Error	Meaning in this system	Consequence
Actual in_uniform missed	Compliant student not detected	Usually harmless for violation monitoring, but lowers recall.

Of the 209 in_uniform instances in the validation set, 138 (66%) were correctly detected, 67 (32%) were misclassified as not_uniform, and 4 (2%) were missed. For the not_uniform class, 195 of 236 instances (83%) were correctly detected, 39 (17%) were misclassified as in_uniform, and 2 (1%) were missed. Confusion between the two classes therefore accounts for 106 detections (67 + 39), and not_uniform was recognized more reliably than in_uniform. The largest counts are in the background column: 496 in_uniform and 1,130 not_uniform predictions did not match any annotated student, and 69% of these false alarms were not_uniform (Figure 4.3). These counts are inflated because Ultralytics validation normally keeps detections down to a very low confidence threshold (default 0.001) when it builds the matrix, so many weak and duplicate boxes are counted. They should be read as a comparison between classes, not as the error rate a user would see at the 0.50 threshold used for demonstration. The most frequent confusion between the two classes was a compliant student predicted as not_uniform (67 cases), which is discussed further in Section 4.5.

4.3 Model Performance

Table 4.5. Overall performance of the selected YOLO11n weights on the validation set (not the held-out test set).

Metric	Result
Precision	0.841
Recall	0.792
F1-score	0.81 (maximum of the all-classes F1 curve, at confidence 0.463)
mAP@0.50	0.866
mAP@0.50:0.95	0.510

Table 4.6. Per-class performance on the validation set

Class	Images	Instances	Precision	Recall	mAP@0.50	mAP@0.50:0.95
in_uniform	61	209	0.911	0.780	0.906	0.550
not_uniform	45	236	0.772	0.803	0.826	0.469
all	75	445	0.841	0.792	0.866	0.510

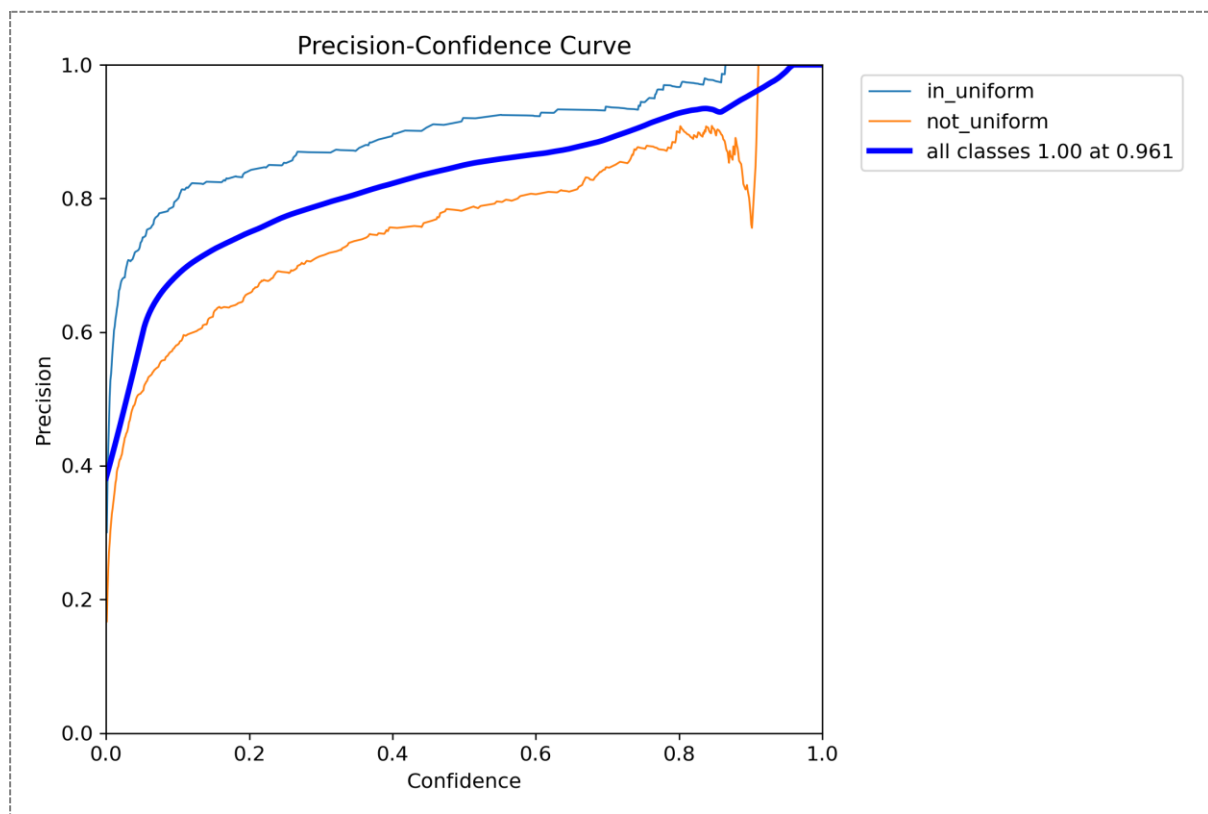


Figure 4.4. Validation precision–recall curve. Precision generally decreases as recall is pushed higher; the reported $\text{mAP}@0.50$ is 0.866 overall (0.906 for `in_uniform` and 0.826 for `not_uniform`).

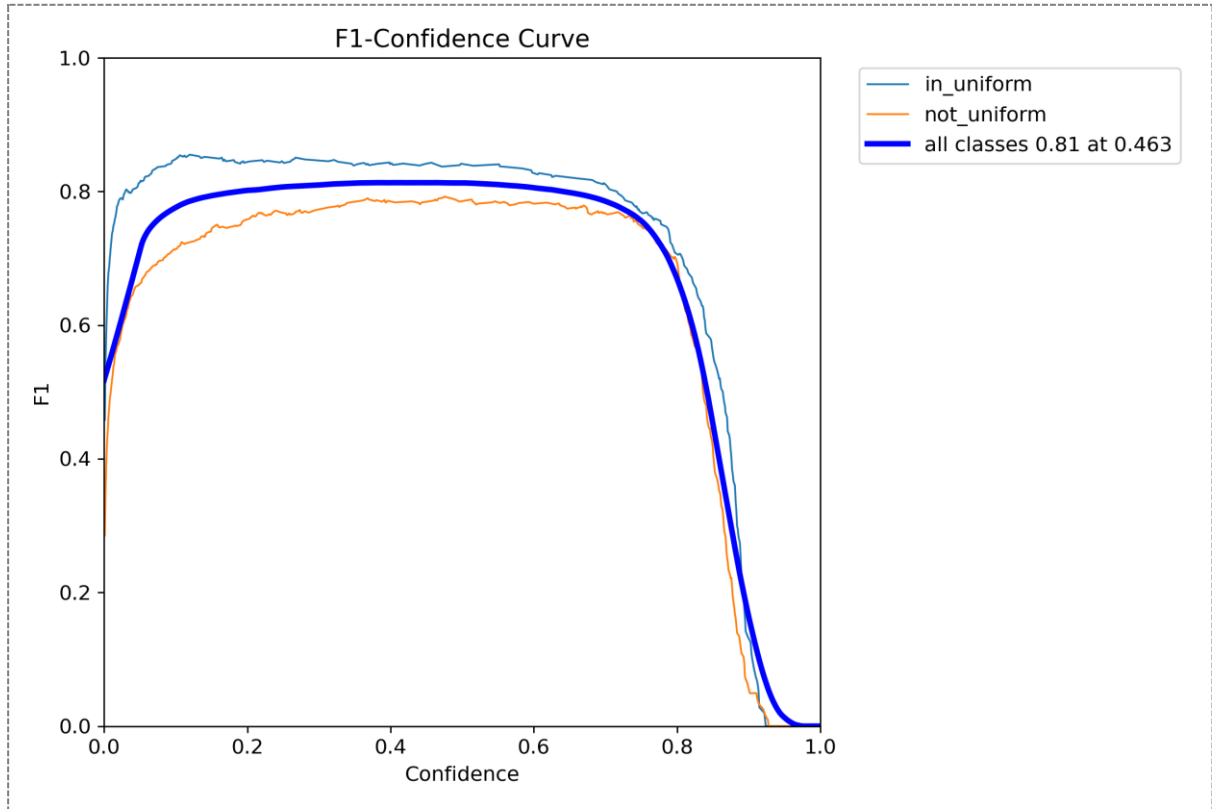


Figure 4.5. Validation F1–confidence curve. The all-class F1 score peaks at approximately 0.81 at a confidence threshold of 0.463.

Precision is the proportion of predicted detections that are correct; low precision means more false alarms. Recall is the proportion of annotated objects that are detected; low recall means more missed objects. The precision–recall curve (Figure 4.4) shows how precision and recall change as the confidence threshold changes. Its area-based summary is average precision (AP), and mAP is the mean AP across classes. $\text{mAP}@0.50$ counts a detection as a match when box IoU is at least 0.50; $\text{mAP}@0.50:0.95$ averages AP over IoU thresholds from 0.50 through 0.95, so it is a stricter measure of localization quality. The model reached a $\text{mAP}@0.50$ of 0.866 (Figure 4.4), with 0.906 for `in_uniform` and 0.826 for `not_uniform`, and a $\text{mAP}@0.50:0.95$ of 0.510 (0.550 and 0.469). The large gap between the two shows that the model usually finds the right student but its boxes are not tightly aligned, which is expected in crowded group photos with overlapping students. These values are lower than the best validation values in Table 4.7 (0.9417 and 0.5935) even though both refer to the same 75 images. Table 4.7 reports the maximum over 154 epochs, which is optimistic (Section 4.3.1), and a re-evaluation of `best.pt` can differ with its evaluation settings, so 0.866 and 0.510 are the values to report. The F1–confidence curve (Figure 4.5) plots the harmonic mean of precision and recall as the confidence threshold changes. The all-class F1 score peaks at approximately 0.81 at confidence 0.463 and remains near that level across a broad threshold range of roughly 0.2–0.7. The demonstration threshold of 0.50 is within this plateau, but the curve does not prove that all classes or deployment conditions will perform equally well. At the F1-optimal threshold, precision and recall are both close to 0.8 (0.841 and 0.792, Table 4.5). The curves also show the usual trade-off: precision reaches 1.00 only at a confidence of

0.961, where almost no objects are still detected, while recall is 0.96 when the threshold is 0. The `in_uniform` class performed better than `not_uniform` on `mAP@0.50` and F1 (about 0.85 against 0.78 on the F1 plateau), and its precision is higher at every confidence level, whereas `not_uniform` has a slightly higher recall at low confidence. Because `not_uniform` has at least as many training instances as `in_uniform` (1,067 against 916 in the full dataset, Table 3.2), the difference is most likely due to appearance: `in_uniform` is one consistent garment, while `not_uniform` covers any other clothing, including blue and white garments that resemble the uniform (Section 4.5).

4.3.1 Controlled Experiment

To investigate how a training parameter affects performance, a controlled experiment was conducted in which only the YOLO model size was changed (YOLO11n, YOLO11s, YOLO11m, YOLO11l, and YOLO11x). The dataset split, image size (736), batch size (16), and random seed (42) were kept identical across runs, and every model followed the same two-phase procedure described in Section 3.4. Tables 4.7 and 4.8 report the best validation values recorded during training. They are not test-set results.

Table 4.7. Effect of YOLO model size on validation performance (two-phase training; best values per metric)

Model	Epochs (head + full)	Precision	Recall	mAP50	mAP50-95	Training time (min)
YOLO11n	154 (58 + 96)	0.9341	0.9126	0.9417	0.5935	11.1
YOLO11s	121 (28 + 93)	0.9213	0.9022	0.9365	0.5868	15.4
YOLO11m	93 (28 + 65)	0.9216	0.9070	0.9416	0.5844	25.2
YOLO11l	74 (30 + 44)	0.9171	0.9026	0.9352	0.5987	25.2
YOLO11x	97 (14 + 83)	0.9282	0.9056	0.9417	0.5929	55.8

Precision, recall, mAP50, and mAP50-95 are the best validation values each model reached during training, and each metric may peak at a different epoch. Training time covers both phases. The validation set has 75 images.

Table 4.8. Contribution of each training phase to the best validation mAP50

Model	Head epochs	Full epochs	Best mAP50 (head)	Best mAP50 (full)	Change
YOLO11n	58	96	0.9363	0.9417	+0.0055
YOLO11s	28	93	0.9298	0.9365	+0.0067
YOLO11m	28	65	0.9289	0.9416	+0.0128
YOLO11l	30	44	0.9352	0.9314	-0.0038
YOLO11x	14	83	0.9280	0.9417	+0.0137

The five model sizes reached similar peak validation scores. Best `mAP@0.50` ranged from 0.9352 (YOLO11l) to 0.9417 (YOLO11n and YOLO11x), while best `mAP@0.50:0.95` ranged from 0.5844 (YOLO11m) to 0.5987 (YOLO11l). YOLO11l had the highest peak

mAP@0.50:0.95 but not the highest mAP@0.50. Each model was trained once with one seed and evaluated against a validation set of only 75 images. Therefore, these small differences do not establish that one size is statistically more accurate; the clearer observed difference is training cost. The epoch-to-epoch swings in the curves are of the same order: YOLO11n's mAP50 ranged from 0.9093 to 0.9290 across the sampled epochs after epoch 68.

What changed clearly was the cost. The average training time per epoch rose from 4.18 s for YOLO11n to 7.52 s for YOLO11s, 16.14 s for YOLO11m, 20.30 s for YOLO11l, and 34.52 s for YOLO11x, and the total training time rose from 11.1 minutes for YOLO11n to 55.8 minutes for YOLO11x, about five times longer for the same best mAP50 of 0.9417. Total times also depend on how many epochs each run lasted (154 for YOLO11n and 74 for YOLO11l), so the per-epoch figure is the fairer comparison. Inference speed was not measured in this experiment.

Table 4.8 shows that unfreezing the backbone improved the best mAP50 for four of the five models, by 0.0055 to 0.0137, with the largest gains for YOLO11m (+0.0128) and YOLO11x (+0.0137). For YOLO11l, the full phase did not beat the head phase (0.9314 against 0.9352). Unfreezing also caused a temporary drop in validation mAP50 that grew with model size in the sampled epochs. The lowest sampled value after unfreezing was 0.8611 for YOLO11n (epoch 59), 0.8374 for YOLO11s (epoch 29), 0.8046 for YOLO11m (epoch 29), 0.7272 for YOLO11l (epoch 35), and 0.5798 for YOLO11x (epoch 15), and each model recovered afterwards.

Taken together, the larger models cost much more training time without a clear accuracy benefit on this dataset, which supports using YOLO11n as the final model. These results should be read with caution. They are validation values from single runs, and the best value of each metric is a maximum over many epochs, so it is optimistic. The last-epoch values were lower, for example 0.9124 mAP50 and 0.5513 mAP50-95 for YOLO11n at epoch 154 (Appendix C.5). A fair comparison of the final models needs the test-set evaluation, ideally repeated with several seeds.

4.4 Error Analysis

The counts in Table 4.12 come from the confusion matrix (Figure 4.2). The causes were then checked on the ground-truth and prediction images of the validation batches (Appendix C.3). Causes marked "observed" were seen in those images; the others are hypotheses. Detections in the batch images are drawn at the validation plotting threshold, so some of the wrong boxes have low confidence (0.3 to 0.4) and would not appear at the 0.50 demonstration threshold, while others have high confidence (0.8 to 0.9).

Table 4.12. Summary of detection errors on the validation set

Error type	Count	Likely cause	Example
False negative: violation (not_uniform) not detected at all	2	Rare (1% of not_uniform instances); probably small or heavily occluded people. Not examined individually.	—

Error type	Count	Likely cause	Example
False positive: compliant student flagged not_uniform	67	Observed: dim lighting with a darker blouse (flagged at 0.9), students partly hidden by hands, chairs, or other people (0.8), and weak boxes (0.3).	Fig. 4.9
Violation predicted as in_uniform (non-uniform garment accepted)	39	Observed: a light purple dress close in color to the light-blue uniform predicted in_uniform (0.38). Other cases were not examined.	Fig. 4.10
Class confusion between in_uniform and not_uniform (rows 2 and 3)	106	Sum of the two rows above; compliant students flagged not_uniform (67) outnumber violations accepted as in_uniform (39).	Fig. 4.9, 4.10
Background false alarms (no matching annotation)	1,626	Observed: duplicate and overlapping boxes in crowds. Also likely: unlabeled or distant people and loose boxes below IoU 0.50. Inflated by the low evaluation threshold (496 in_uniform + 1,130 not_uniform).	Fig. C.2 to C.4



Figure 4.9. Examples of class-confusion errors, including compliant students predicted as not_uniform.

Among the 106 off-diagonal class confusions in the validation confusion matrix, the more frequent error was an actual in_uniform instance predicted as not_uniform: 67 cases (63%). The reviewed examples associate these errors with dim lighting, crowding, and partial occlusion (Figure 4.9). This is a class-confusion count at the validation plotting threshold, not a false-alert rate at the 0.50 demonstration threshold. The in_uniform class had the lower recall in the confusion matrix (66% against 83% for not_uniform), which fits these causes: a uniform that is dark, shadowed, or partly covered looks less like the typical light-blue shirt. Violations accepted as in_uniform (39 cases, 17% of not_uniform instances) involved garments close in color to the uniform (Figure 4.10).

Limitations. The validation set is small (75 images and 445 annotated instances). A verified held-out test-set evaluation is not included in the supplied results, so performance on unseen test images remains unreported. The dataset's collection sources and diversity also need fuller

documentation. The model considers only the upper uniform, and confusion-matrix counts were produced at a low confidence threshold; they should not be interpreted as the error rate at the 0.50 demonstration threshold. Add: whether the model was tested on webcam or CCTV footage.

Implication for use. Because a wrong not_uniform alert could unfairly flag a student, the model's output should be treated as decision support, with a human checking each flagged case before any action is taken.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This project developed a YOLO11n object-detection prototype for identifying whether a student's traditional upper uniform appears compliant (`in_uniform`) or non-compliant (`not_uniform`). The dataset contains 504 images, split into 379 training, 75 validation, and 50 test images. During training, the best recorded validation values were precision = 0.9341, recall = 0.9126, mAP@0.50 = 0.9417, and mAP@0.50:0.95 = 0.5935; these are peak validation values across epochs and should not be presented as held-out test performance. A separate reported evaluation of the selected weights on the validation set yielded precision = 0.841, recall = 0.792, F1 = 0.81, mAP@0.50 = 0.866, and mAP@0.50:0.95 = 0.510. These values are validation results, not test-set results. The report does not contain verified test-set metrics, so generalization to unseen test images cannot yet be quantified.

The controlled experiment, in which five YOLO11 sizes (n, s, m, l, and x) were trained under identical conditions, showed almost the same validation accuracy across sizes (best mAP50 of 0.9352 to 0.9417 and best mAP50-95 of 0.5844 to 0.5987) while the training time per epoch grew from 4.18 s to 34.52 s, so the larger models gave no clear benefit on this dataset. Error analysis showed that the main error was compliant students flagged as `not_uniform` in dim, crowded, or partly occluded scenes (67 cases), followed by violations accepted as `in_uniform` when a garment resembled the uniform in color (39 cases). Overall, the results support the feasibility of using YOLO to assist uniform compliance monitoring at NU Lipa, provided that adequate lighting and human verification of flagged cases are ensured.

Table 5.1. Achievement of project objectives

Objective (from Section 1.3)	Finding / evidence	Section
Explain the fundamental concepts of YOLO and object detection relevant to uniform compliance monitoring.	Computer vision, object detection, YOLO, CNNs, dataset annotation, and the tools used are explained in Chapter 2.	2.1 to 2.7
Identify uniform compliance detection as a real-world object detection problem at NU Lipa and define the target classes.	Manual uniform checking was identified as the problem, and the two classes were defined from Sections I.E and VI.D of the Student Handbook.	1.1, 1.2, B.2
Collect, select, and prepare an image dataset of at least 500 images with at least two classes, divided into training, validation, and test sets.	504 images with two classes were divided into 379 training, 75 validation, and 50 test images.	3.2, App. B
Annotate the images with bounding boxes and class labels compatible with YOLO using CVAT and Roboflow.	Boxes and class labels were drawn in CVAT and exported in YOLO format.	3.3, App. E

Objective (from Section 1.3)	Finding / evidence	Section
Configure and train a YOLO model using Python and the Ultralytics library.	Five YOLO11 sizes were trained with a two-phase procedure (head, then full). The final model, YOLO11n, trained for 154 epochs in 11.1 minutes.	3.4, 3.5, 4.1
Evaluate the trained model using precision, recall, mAP@0.50, mAP@0.50:0.95, and a confusion matrix.	Validation metrics were reported for all five sizes (best mAP50 of 0.9352 to 0.9417). Final model on the validation set: precision 0.841, recall 0.792, mAP@0.50 0.866, mAP@0.50:0.95 0.510, confusion matrix and curves; test-set evaluation	4.2, 4.3
Test the model on unseen images, video, or webcam input.		4.4
Analyze the strengths, limitations, and errors of the developed model.	The model-size and training-phase comparison is complete. Error analysis: confusion counts and visual review of validation batches (Section 4.5)	4.3.1, 4.5
Document the development process in a technical report and present a working demonstration.	The development process is documented in this report.	All, App. E

5.2 Recommendations

Based on the results and the error analysis, the following improvements are recommended:

- **More and more varied training images:** add images across times of day, gates and corridors, camera heights, crowd levels, and body sizes, especially for conditions where a student is wearing a jacket and where an item such as a bag strap or even a hand covers the nu logo of the uniform.
- **Improved annotation:** write a one-page labeling guide, have a second person audit a sample of labels, and re-label ambiguous cases such as partly covered uniforms and garments close in color to the uniform .
- **Class balancing:** the classes are fairly balanced (916 in_uniform and 1,067 not_uniform instances, Table 3.2), so balancing is not the first priority. If later data collection changes the ratio, collect more of the smaller class or apply targeted augmentation.
- **Data augmentation:** use brightness, contrast, blur, and scale changes that match campus conditions; avoid unrealistic transforms such as vertical flips.
- **Hyperparameter tuning:** tune learning rate, image size, and augmentation strength using the validation set only.
- **Training schedule:** the head-only phase alone reached a best mAP50 of 0.9280 to 0.9363, and the full phase added at most 0.0137, while the full

phase caused a large temporary drop for the larger models. A lower learning rate or a shorter full phase could be tested to save training time.

- **Alternative model sizes:** YOLO11s, YOLO11m, YOLO11l, and YOLO11x were compared with YOLO11n (Section 4.3.1) and gave no clear accuracy gain for much longer training. Larger models are worth revisiting only after the dataset grows, and the comparison should be repeated with several seeds so that small differences can be separated from noise.
- **Additional classes:** extend beyond the upper uniform (e.g., improper wearing, ID, lower uniform) as future scope.
- **Real-time deployment:** connect to a CCTV/RTSP stream, add object tracking so one student is not counted repeatedly, and test on the target edge device.
- **Human verification and privacy:** keep a staff member in the loop for every flagged case, avoid storing identifiable images longer than needed, and secure approval from the school administration. Review Republic Act No. 10173 (Data Privacy Act of 2012) with the school's data privacy officer before any real deployment.

APPENDIX A

APPENDIX B DATASET INFORMATION

B.1 Dataset Summary

Table B.1. Dataset summary

Item	Description
Dataset name	unidatset
Purpose	Detect proper wearing of the traditional upper uniform at NU Lipa
Source	Public images self collected
Collection dates and locations	Campus off campus day and night
Capture device	unkown
Total images	504
Classes	2: in_uniform, not_uniform
Annotation tool and format	CVAT ; YOLO .txt bounding boxes
Preprocessing / augmentation applied before training	'hsv_h': 0.015, 'hsv_s': 0.7, 'hsv_v': 0.4, 'degrees': 5.0, 'translate': 0.1, 'scale': 0.5, 'shear': 0.0, 'fliplr': 0.5, 'flipud': 0.0, 'mosaic': 1.0, 'mixup': 0.1,

B.2 Class Definitions and Annotation Rules

Table B.2. Class definitions

Class	Definition	Examples
in_uniform	Student wearing the prescribed upper uniform properly,	collar, buttons, tucked/untucked rule, sleeves, ID if required
not_uniform	Student not wearing the prescribed upper uniform, or wearing it improperly	civilian shirt, hoodie over uniform, uniform unbuttoned

B.3 Train / Validation / Test Split

Table B.3. Dataset split

Split	Share	Images
Training	75%	379
Validation	15%	75
Test	10%	50
Total	100%	504

B.4 Image Quality and Diversity

Table B.4. Image conditions in the dataset

Condition	Variation covered	Approx. images
Lighting	bright outdoor / indoor / low light / backlit	100
Camera angle	frontal / side	100
Distance	near / mid	100
Crowd density	single / small group / crowded	100

B.5 Data Cleaning and Preparation

- Removed ___ blurred, corrupt, or duplicate images.
- Checked label files: every image has a matching .txt; class IDs are 0 or 1; coordinates normalized between 0 and 1.
- Fixed ___ annotation errors found during review.

APPENDIX C TRAINING RESULTS

C.1 Files Produced by Training

Table C.1. Training output files (results/yolo_training/)

File	Description
weights/best.pt, weights/last.pt	Best checkpoint (by validation fitness) and final-epoch checkpoint
results.csv / results.png	Per-epoch losses and metrics, and the plotted curves
confusion_matrix.png / confusion_matrix_normalized.png	Confusion matrices on the validation split
PR_curve.png (BoxPR_curve.png in newer versions), BoxP_curve.png, BoxR_curve.png, BoxF1_curve.png	Precision–recall, precision–confidence, recall–confidence, and F1–confidence curves
labels.jpg	Distribution of instances and box sizes in the training labels
train_batch*.jpg, val_batch*_labels.jpg, val_batch*_pred.jpg	Sample training mosaics and validation labels versus predictions
args.yaml	Exact training arguments used

C.2 Epoch-by-Epoch Results (from results.csv)

Table C.2. Selected epochs

Epoch	train/box_loss	train/cls_loss	val/box_loss	val/cls_loss	Precision	Recall	mAP50	mAP50-95
1	1.9521	3.3393	1.3636	3.5731	0.0092	0.5672	0.1073	0.0358
129 (best mAP 50-95)	0.9960	0.6166	1.2015	0.6032	0.8772	0.8698	0.9222	0.5935
154 (last)	0.9677	0.5882	1.2632	0.6552	0.8809	0.8277	0.9124	0.5513

C.3 Additional Training Figures

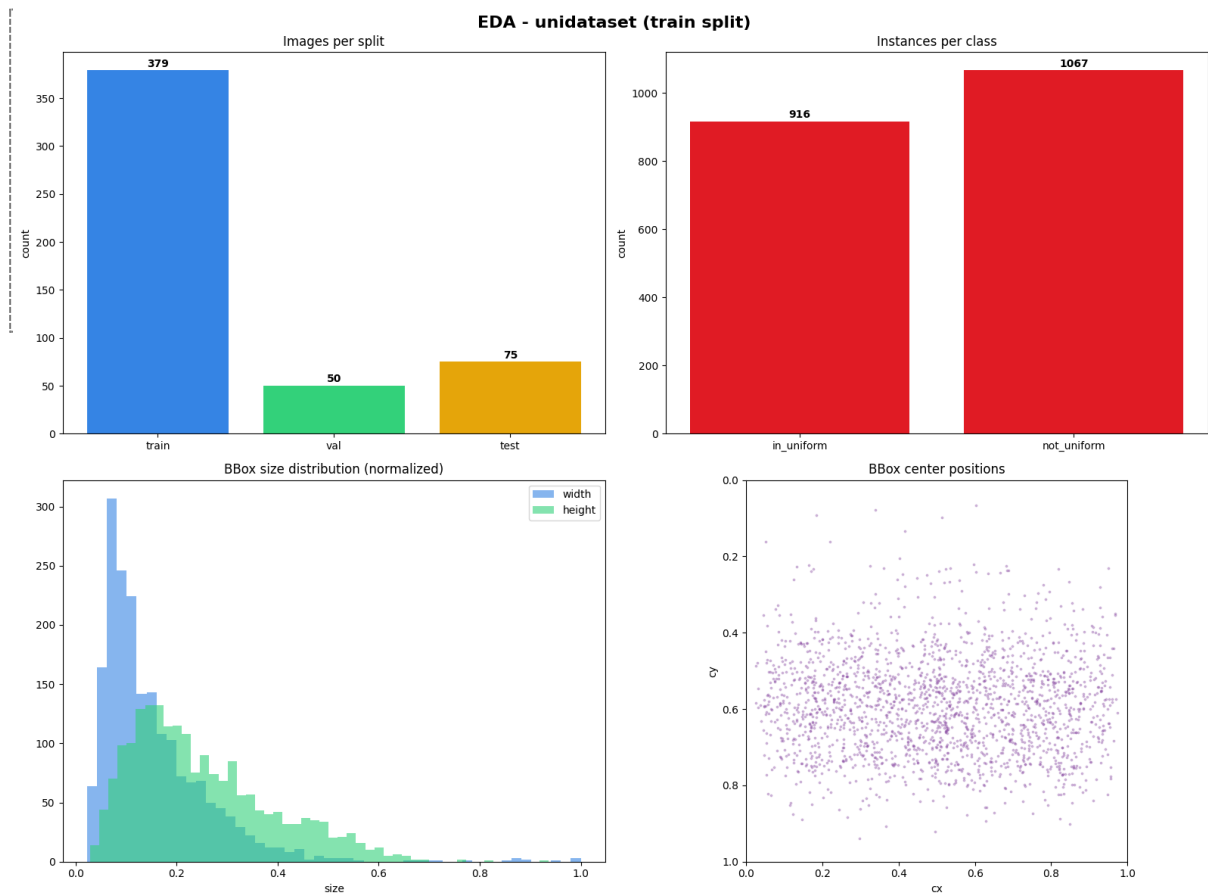


Figure C.1. Label distribution of the training set (labels.jpg).



Figure C.2. Validation batch 0 (val_batch0): ground-truth labels (top) versus model predictions (bottom).



Figure C.4. Validation batch 2 (val_batch2): ground-truth labels (top) versus model predictions (bottom).

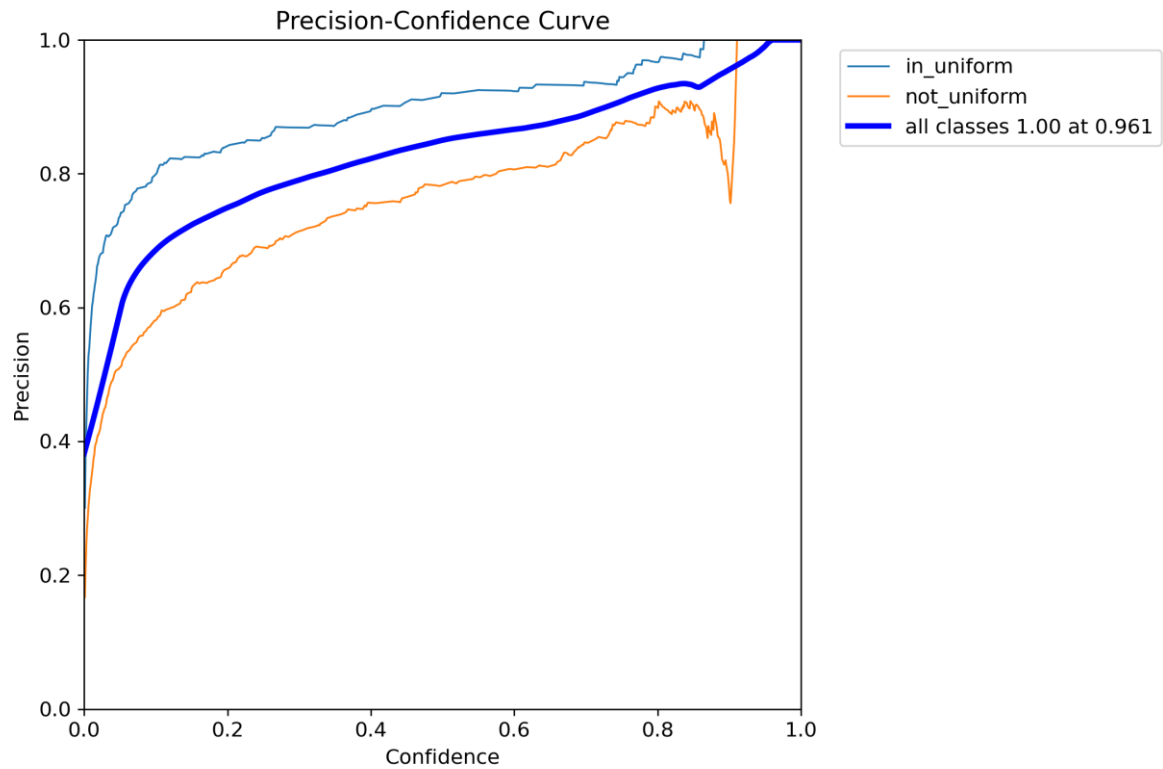


Figure C.5. Precision–confidence curve of the final model (all classes 1.00 at a confidence of 0.961).

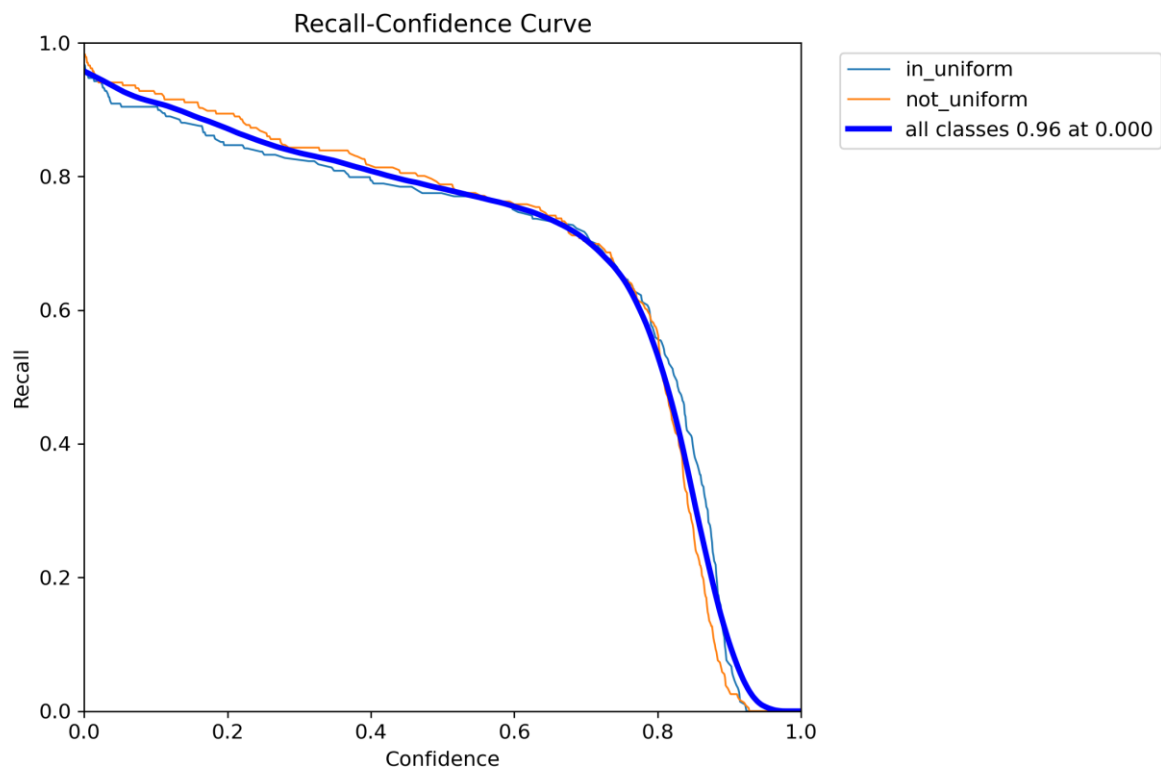


Figure C.6. Recall–confidence curve of the final model (all classes 0.96 at a confidence of 0).

C.4 Experiment Outputs and Console Log

Raw experiment results are saved in `results/evaluation/experiments.csv`. The console output of the final evaluation (validation split) is pasted below.

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)
all	75	445	0.841	0.792	0.866	0.51
in_uniform	61	209	0.911	0.78	0.906	0.55
not_uniform	45	236	0.772	0.803	0.826	0.469

Overall metrics:

Precision : 0.8413
 Recall : 0.7918
 mAP@0.5 : 0.8661
 mAP@0.5:0.95 : 0.5098

Per-class mAP@0.5:0.95:

in_uniform : 0.5504
 not_uniform : 0.4692

C.5 Training Results of All Model Sizes

Tables C.3 to C.5 summarize the training runs of the five YOLO11 sizes compared in Section 4.3.1. All values are validation results from the training logs, with both

training phases counted as one session. Tables C.6 and C.7 give the sampled epoch-by-epoch progression and the loss summary of the chosen model, YOLO11n.

Table C.3. Best validation values (with the epoch where each was reached) and last-epoch values

Model	Best mAP50 (epoch)	Best mAP50-95 (epoch)	Best precision (epoch)	Best recall (epoch)	Last mAP50	Last mAP50-95
YOLO11n	0.9417 (121)	0.5935 (129)	0.9341 (112)	0.9126 (42)	0.9124	0.5513
YOLO11s	0.9365 (72)	0.5868 (96)	0.9213 (99)	0.9022 (72)	0.8948	0.5369
YOLO11m	0.9416 (68)	0.5844 (68)	0.9216 (62)	0.9070 (18)	0.9067	0.5596
YOLO11l	0.9352 (20)	0.5987 (49)	0.9171 (22)	0.9026 (28)	0.9175	0.5584
YOLO11x	0.9417 (59)	0.5929 (72)	0.9282 (60)	0.9056 (90)	0.9268	0.5829

Table C.4. Training and validation loss values at the last epoch

Model	Train box	Train cls	Train DFL	Val box	Val cls	Val DFL
YOLO11n	0.9677	0.5882	1.0708	1.2632	0.6552	1.2537
YOLO11s	0.8291	0.4807	1.0405	1.2969	0.6793	1.3798
YOLO11m	0.8978	0.5205	1.1174	1.3131	0.6996	1.5306
YOLO11l	1.0437	0.6052	1.2480	1.2601	0.6781	1.5297
YOLO11x	1.0055	0.5732	1.2723	1.2667	0.6623	1.6407

Table C.5. Training time of each model size

Model	Total epochs	Time to best mAP50 (min)	Average time per epoch (s)	Total time (min)
YOLO11n	154	8.7	4.18	11.1
YOLO11s	121	8.9	7.52	15.4
YOLO11m	93	17.7	16.14	25.2
YOLO11l	74	5.4	20.30	25.2
YOLO11x	97	33.3	34.52	55.8

Table C.6. Sampled epoch-by-epoch progression of YOLO11n (head phase: epochs 1 to 58, full phase: epochs 59 to 154; validation values, from the training log)

Epoch	Phase	mAP50	mAP50-95	Precision	Recall	Train loss	Val loss
1	Head	0.1073	0.0358	0.0092	0.5672	7.2582	6.4814
12	Head	0.8301	0.5085	0.7381	0.7634	3.8499	3.5895
23	Head	0.8572	0.5242	0.7454	0.8440	3.4182	3.5747
34	Head	0.9137	0.5511	0.8718	0.8225	3.3133	3.3306
59	Full	0.8611	0.5121	0.7639	0.8064	3.3892	3.3214
68	Full	0.9160	0.5595	0.8388	0.8582	3.1336	3.2195
77	Full	0.9232	0.5640	0.8422	0.8620	3.0389	3.2840
86	Full	0.9093	0.5543	0.8871	0.8638	2.9220	3.2348

Epoch	Phase	mAP50	mAP50-95	Precision	Recall	Train loss	Val loss
95	Full	0.9144	0.5547	0.9059	0.8110	2.7700	3.1708
104	Full	0.9254	0.5645	0.8964	0.8328	2.7638	3.1204
113	Full	0.9123	0.5739	0.8512	0.8823	2.7823	3.0653
122	Full	0.9290	0.5847	0.8866	0.8396	2.6689	3.0611
131	Full	0.9125	0.5686	0.8803	0.8760	2.6608	3.2278
140	Full	0.9095	0.5686	0.8805	0.8578	2.5739	3.1360
154	Full	0.9124	0.5513	0.8809	0.8277	2.6268	3.1720

Table C.7. Training and validation loss values of YOLO11n at the start, after the head phase, and at the end of training

Loss type	Initial	After head phase	Final	Min	Max
Box loss (train)	1.9521	1.1624	0.9677	0.8903	1.9521
Class loss (train)	3.3393	0.7622	0.5882	0.5237	3.3393
DFL loss (train)	1.9668	1.1637	1.0708	1.0312	1.9668
Box loss (val)	1.3636	1.2429	1.2632	1.2014	1.5985
Class loss (val)	3.5731	0.6897	0.6552	0.5944	3.5731
DFL loss (val)	1.5447	1.1965	1.2537	1.1877	1.5447

APPENDIX E SCREENSHOTS

Screenshots documenting the development process. The annotation screenshots (E.1 and E.2) are explicitly required by the project guideline.

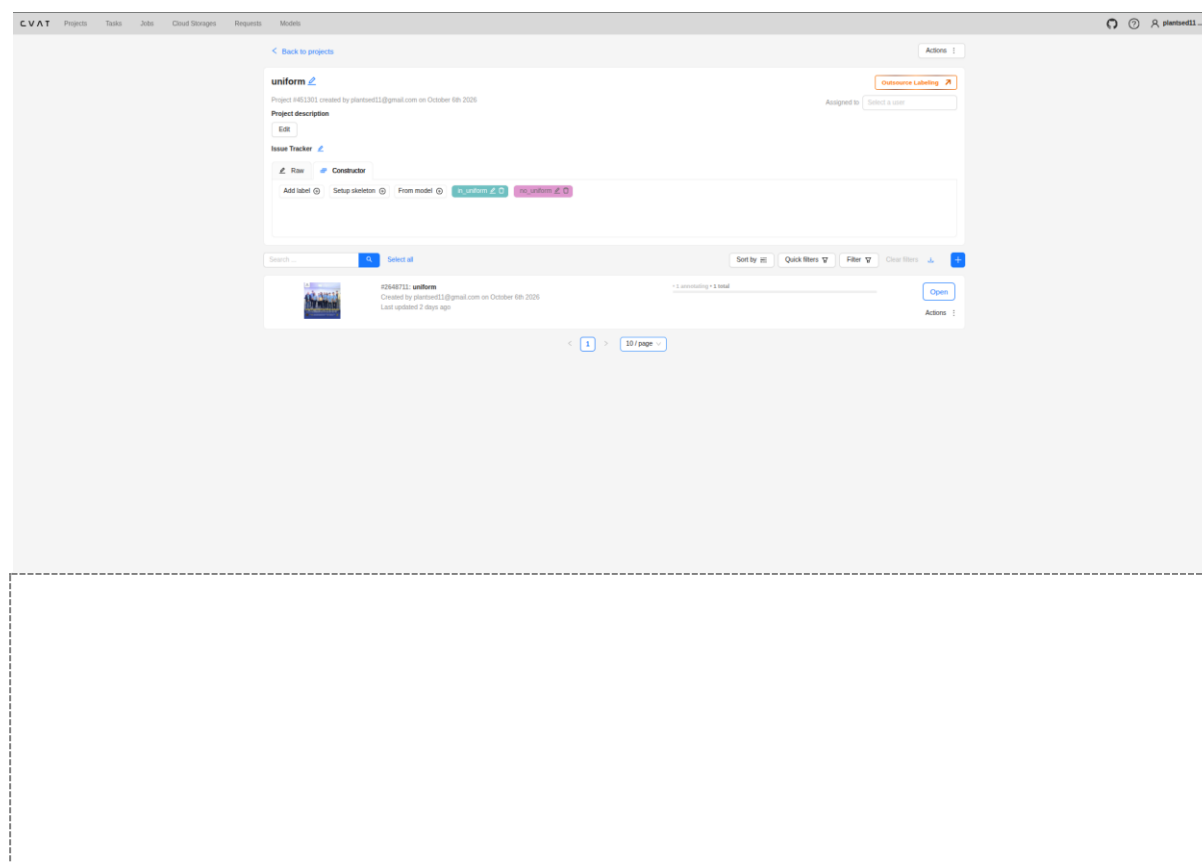


Figure E.1. Annotation tool interface showing the project and image list.

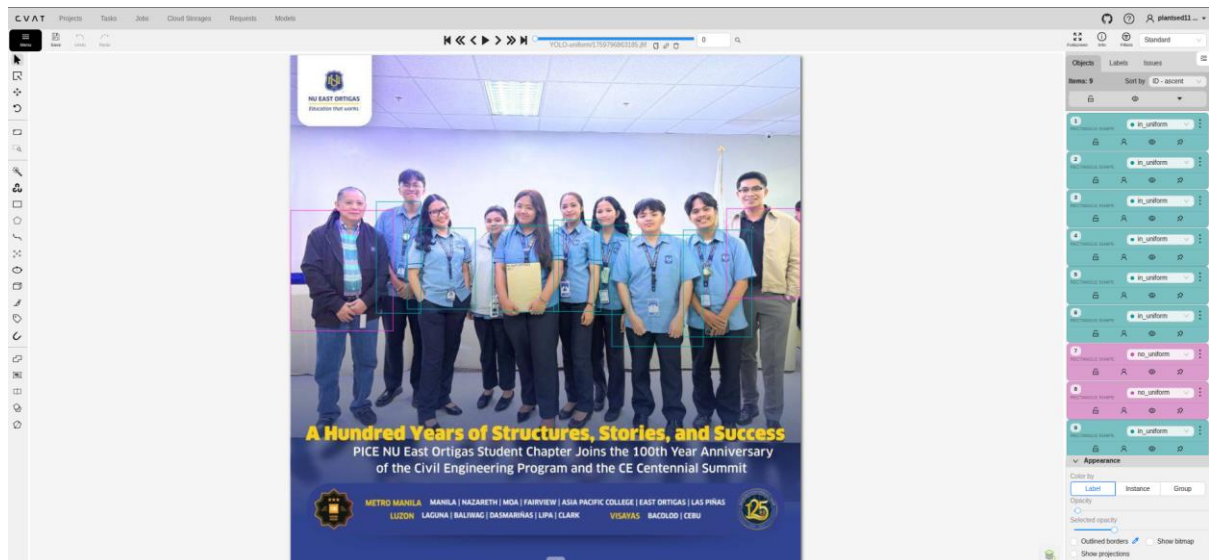


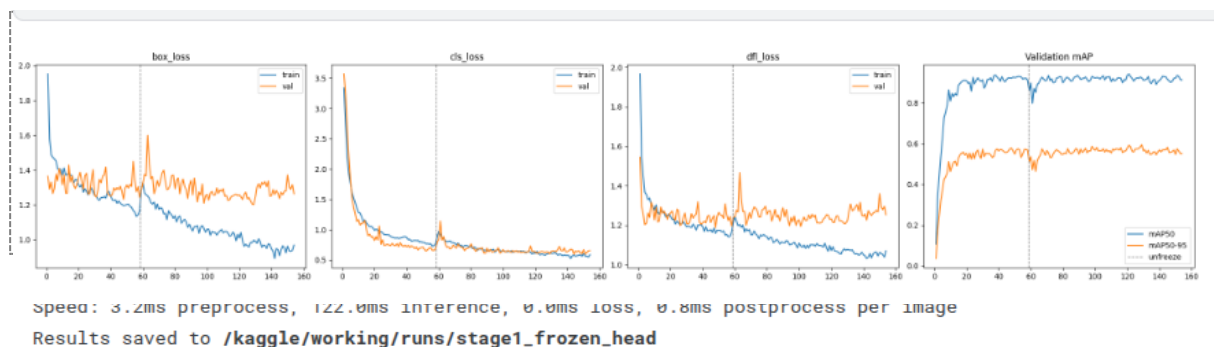
Figure E.2. Example of bounding boxes and class labels (in_uniform, not_uniform) being drawn.

```
print("Python version:", __import__("sys").version)
print("Ultralytics version:", ultralytics.__version__)
print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

Python version: 3.13.15 (main, Aug 6 2026, 11:06:22) [GCC 13.3.0]
 Ultralytics version: 8.4.174
 PyTorch version: 2.11.0+cu128
 CUDA available: True



Figure E.3. Environment check (yolo checks) showing Python, PyTorch, and GPU/CPU.



```

      Class      Images  Instances   Box(P          R      mAP50  mAP50-95): 100%
----- 5/5 1.3it/s 3.9s1.1ss
      all         75       445     0.841     0.792     0.866     0.51
    in_uniform     61       209     0.911     0.78      0.906     0.55
    not_uniform     45       236     0.772     0.803     0.826     0.469
Speed: 5.1ms preprocess, 10.7ms inference, 0.0ms loss, 10.7ms postprocess per image
Results saved to /kaggle/working/runs/detect/val-3

Overall metrics:
  Precision      : 0.8413
  Recall         : 0.7918
  mAP@0.5        : 0.8661
  mAP@0.5:0.95   : 0.5098

Per-class mAP@0.5:0.95:
  in_uniform     : 0.5504
  not_uniform     : 0.4692

```

Figure E.4. Training console output showing epochs, losses, and final metrics.

WARNING  imgsiz=[763] must be multiple of max stride 32, updating to [768]

Predictions (test split, conf=0.37)



Prediction grid saved to /kaggle/working/test_predictions.png

Figure E.5. Evaluation console output on the validation set.

Figure E.6. Live demonstration window (webcam / video / image) with detections.

APPENDIX F

REFERENCES

Documentation note: Entries below were compiled from search-result pages. Verify authors, years, volume/page numbers, and DOIs against the papers, and fill the highlighted gaps. Add the references you cited in Chapters 1–3 and remove any you did not cite. Entries are in APA 7 style, alphabetical.

- Basukala, B., Suwal, B., Bakhunchhe, S., & Duwal, U. (2025). Student uniform detection with face recognition [Bachelor's thesis, Khwopa Engineering College]. <https://elibrary.khec.edu.np/handle/123456789/879>
- [Authors]. ([Year]). Detection and classification of dress code violations in educational environments using deep learning. [Journal: AJERD, volume(issue), pages]. <https://journals.abuad.edu.ng/ajerd/article/view/1362>
- [Authors]. ([Year]). Development of student's uniform compliance detection. [Proceedings of ICDSOS, STIS]. <https://proceedings.stis.ac.id/icdsos/article/download/298/93>
- [Authors]. (2026). Peningkatan akurasi sistem dress-code surveillance [Improving the accuracy of a dress-code surveillance system]. Bulletin of Information Technology (BIT), 7(2), 222–231. <https://journal.fkpt.org/index.php/BIT/article/download/2760/1200/>
- gisketch. ([n.d.]). complete-uniform-detection-YOLOv8 [Source code; software accompanying the paper on a YOLOv8 uniform detection software for University of Southern Mindanao students]. GitHub. <https://github.com/gisketch/complete-uniform-detection-YOLOv8>
- Jocher, G., Qiu, J., & Chaurasia, A. (2023). Ultralytics YOLO (Version 8.0.0) [Computer software]. <https://github.com/ultralytics/ultralytics>
- Lam, K. N., La, T. T., Nguyen, K. D., Le, M. M., Truong, V. T., & Ware, A. (2023). Monitoring attendance and checking school uniforms using YOLOv8. In Communications in Computer and Information Science (pp. 200–207). ISDS 2023. [publisher / DOI]
- [NU Lipa]. ([Year]). [Student handbook] ([section on uniform policy]). National University Lipa.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 779–788. <https://doi.org/10.1109/CVPR.2016.91>
- Republic Act No. 10173. Data Privacy Act of 2012 (Philippines).

Tantra, B. J., & Widjaja, M. (2025). Automatic detection of dress-code surveillance in a university using YOLO algorithm. *IAES International Journal of Artificial Intelligence*, 14(2), 1568–1575. <https://doi.org/10.11591/ijai.v14.i2.pp1568-1575>

Ultralytics. (n.d.-a). Ultralytics YOLO11. Ultralytics Docs. <https://docs.ultralytics.com/models/yolo11>

Ultralytics. (n.d.-b). YOLO performance metrics. Ultralytics Docs. <https://docs.ultralytics.com/guides/yolo-performance-metrics>